

O Caso Lojas Americanas

Inconsistências contábeis, governança corporativa
e implicações para a transparência financeira

“Em janeiro de 2023, uma das maiores varejistas do Brasil revelou inconsistências contábeis estimadas em aproximadamente R\$ 40 bilhões, desencadeando uma das mais relevantes crises de governança corporativa do país.”

Autor: Israel Junior,
Análise de Demonstrações Financeiras, Auditoria, Ética nos Negócios

Tempo estimado de discussão: 90 a 120 minutos

Sumário

Objetivos de Aprendizagem	2
1 Contexto e Apresentação do Caso	3
2 O Núcleo do Problema: o Tratamento Contábil do “Risco Sacado”	4
3 Consequências e Repercussões	5
3.1 Impactos de Mercado	5
3.2 Resposta Institucional e Regulatória	5
3.3 Efeitos sobre a Cadeia de Valor	5
4 Análise: Limitações do Modelo Tradicional	6
5 Perspectivas Tecnológicas: Registros Distribuídos e Auditoria Contínua	7
6 Blockchain como Resposta ao Problema do Risco Sacado	7
6.1 Conceitos Fundamentais	7
6.2 Aplicação Direta ao Caso	8
6.3 Limites e Pré-Condições	9
6.4 Síntese: Tecnologia como Condição Facilitadora	9
7 Considerações Finais	11
A Código-Fonte do Contrato <i>Supply Chain Finance</i>	12
A.1 Visão Geral da Arquitetura	12
A.2 Código-Fonte Completo	14
B Tutorial de Execução do Contrato no Remix	18
B.1 Como funciona a interface do Remix	18
B.2 Passo 1 — Anotar os endereços das contas	18
B.3 Passo 2 — Autorizar o banco	19
B.4 Passo 3 — Criar a fatura	19
B.5 Passo 4 — Empresa aprova a fatura	20
B.6 Passo 5 — Banco financia a fatura	20
B.7 Passo 6 — Empresa liquida a dívida	20
B.8 Passo 7 — Verificar o resultado	21
B.9 Resumo: qual conta usar em cada passo	21
B.10 Conexão com o caso de estudo	22

Objetivos de Aprendizagem

Objetivos de Aprendizagem

Ao final da discussão deste caso, espera-se que os participantes sejam capazes de:

1. **Compreender** a natureza econômica e o tratamento contábil de operações de financiamento de fornecedores (*risco sacado* ou *reverse factoring*), distinguindo dívida financeira de obrigações comerciais.
2. **Analisar criticamente** as fragilidades estruturais dos mecanismos tradicionais de governança corporativa, controles internos e auditoria externa diante de operações financeiras complexas.
3. **Avaliar** o impacto de classificações contábeis inadequadas sobre indicadores de alavancagem, percepção de risco e tomada de decisão de investidores e credores.
4. **Identificar** os papéis e as responsabilidades dos diferentes atores envolvidos na produção e validação de informações financeiras: gestores, conselho de administração, comitê de auditoria, auditores independentes e órgãos reguladores.
5. **Discutir** as implicações sistêmicas de falhas informacionais em cadeias de valor complexas, considerando os efeitos sobre fornecedores, instituições financeiras e o mercado de capitais.
6. **Refletir** sobre o potencial e os limites de tecnologias emergentes — particularmente *blockchain* e registros distribuídos — como mecanismos para mitigar assimetrias de informação, reduzir discricionariedade contábil e viabilizar auditoria contínua.
7. **Distinguir** condições facilitadoras (tecnológicas) de condições suficientes (institucionais e regulatórias) na construção de modelos robustos de transparência corporativa.
8. **Formular** recomendações práticas de aprimoramento de controles, governança e divulgação aplicáveis a companhias abertas brasileiras.

1 Contexto e Apresentação do Caso

O episódio envolvendo a Lojas Americanas, revelado publicamente em janeiro de 2023, constitui um dos mais relevantes casos recentes de falha de governança corporativa no Brasil. A companhia divulgou a existência de inconsistências contábeis estimadas em aproximadamente R\$ 40 bilhões, desencadeando uma crise de confiança que resultou em forte desvalorização de suas ações, além do subsequente pedido de recuperação judicial.

O caso ganhou ampla repercussão nacional e internacional, não apenas pelo montante envolvido, mas também pelas fragilidades estruturais que expôs nos mecanismos de controle e transparência contábil. Tradicionalmente percebida como uma empresa sólida, com presença consolidada no varejo brasileiro e acionistas de referência altamente reputados, a Lojas Americanas viu sua credibilidade rapidamente erodida diante das revelações.

A Americanas usava muito risco sacado com seus fornecedores. O escândalo de janeiro de 2023 foi sobre um problema contábil específico: a empresa registrava as dívidas do risco sacado como "fornecedores a pagar" no balanço, em vez de "dívida bancária". Por que isso importou? Porque "fornecedores a pagar" é uma dívida operacional normal, enquanto "dívida bancária" pesa muito mais no endividamento da empresa aos olhos do mercado. Quando essa diferença veio à tona, falou-se em "inconsistências contábeis" de cerca de R\$ 40 bilhões, daí a recuperação judicial e todo o caos que se seguiu.

2 O Núcleo do Problema: o Tratamento Contábil do “Risco Sacado”

O núcleo do problema estava associado ao tratamento contábil de operações de financiamento de fornecedores, conhecidas como **risco sacado** (*reverse factoring*). Nesse arranjo, instituições financeiras antecipam pagamentos a fornecedores, assumindo o direito de crédito junto à empresa compradora.

Como funciona o risco sacado, em síntese:

1. A empresa compradora adquire mercadorias de um fornecedor, gerando uma obrigação a pagar.
2. O fornecedor, em vez de aguardar o prazo original de pagamento, antecipa o recebimento junto a um banco parceiro indicado pela compradora.
3. O banco assume o direito de crédito e passa a cobrar diretamente da empresa compradora no vencimento original (ou estendido).
4. A empresa compradora, agora, deve ao banco — não mais ao fornecedor.

Sob a perspectiva econômica, tais operações configuram **dívida financeira**, uma vez que implicam obrigação futura da empresa perante uma instituição financeira, frequentemente acompanhada de custos financeiros embutidos. Entretanto, no caso em questão, há evidências de que esses passivos foram registrados como **obrigações comerciais** (contas a pagar a fornecedores), e não como dívida financeira propriamente dita.

Essa classificação inadequada contribuiu para subestimar o nível de alavancagem da companhia e distorcer a percepção de risco por parte de investidores e credores. Indicadores amplamente utilizados na análise de crédito — como dívida líquida sobre EBITDA, cobertura de juros e estrutura de capital — passaram a refletir uma imagem mais favorável do que aquela compatível com a realidade econômica das operações.

3 Consequências e Repercussões

As consequências do episódio foram expressivas e multifacetadas, abrangendo dimensões de mercado, institucionais e sistêmicas.

3.1 Impactos de Mercado

Do ponto de vista de mercado, observou-se uma abrupta destruição de valor, refletida na queda acentuada das ações e na deterioração da confiança dos investidores. Debêntures e demais títulos de dívida emitidos pela companhia também sofreram severas desvalorizações, com reflexos relevantes em fundos de investimento e carteiras institucionais.

3.2 Resposta Institucional e Regulatória

No âmbito institucional, o caso motivou investigações conduzidas por órgãos como a **Comissão de Valores Mobiliários (CVM)** e a **Polícia Federal**, com foco na apuração de responsabilidades e na verificação de possíveis irregularidades na divulgação de informações financeiras. A atuação dos auditores independentes, do comitê de auditoria e da alta administração passou a ser objeto de escrutínio detalhado.

3.3 Efeitos sobre a Cadeia de Valor

Bancos credores e fornecedores passaram a enfrentar elevado grau de incerteza quanto à recuperação de seus créditos, evidenciando os efeitos sistêmicos de falhas informacionais em cadeias de valor complexas. Pequenos e médios fornecedores, em particular, foram especialmente afetados pelo risco de descontinuidade de pagamentos e pela contração súbita do crédito comercial no setor varejista.

4 Análise: Limitações do Modelo Tradicional

Sob uma perspectiva analítica, o caso evidencia limitações inerentes ao modelo tradicional de contabilidade e auditoria, caracterizado por:

- **Processos *ex-post*:** a maior parte das verificações ocorre após a consumação dos eventos econômicos, dificultando a detecção tempestiva de irregularidades.
- **Forte dependência de declarações internas:** auditores e reguladores trabalham, em larga medida, sobre informações fornecidas pela própria companhia auditada.
- **Assimetria de informação:** gestores possuem informação substancialmente superior à de investidores, credores e demais *stakeholders*.
- **Discrecionabilidade contábil:** normas baseadas em princípios podem permitir interpretações distintas para operações economicamente equivalentes.

A possibilidade de reclassificação contábil de operações economicamente equivalentes a dívida demonstra como estruturas informacionais opacas podem comprometer a qualidade dos relatórios financeiros e dificultar a avaliação precisa da saúde econômico-financeira de uma organização.

5 Perspectivas Tecnológicas: Registros Distribuídos e Auditoria Contínua

Nesse contexto, o episódio suscita reflexões relevantes sobre a necessidade de mecanismos que promovam maior **transparência**, **rastreabilidade** e **verificabilidade** das informações contábeis.

Abordagens baseadas em registros imutáveis e compartilhados, como aquelas viabilizadas por tecnologias de registro distribuído (*distributed ledger technologies*), podem contribuir para reduzir a discricionariedade na classificação de transações e possibilitar auditoria em tempo real. Ao registrar eventos econômicos de forma sequencial, transparente e verificável, tais sistemas tendem a mitigar riscos de manipulação informacional e a alinhar incentivos entre os diversos participantes da cadeia.

Implicações conceituais: um registro compartilhado entre empresa, fornecedores, bancos e auditores poderia, em tese, tornar inviável o registro de uma mesma operação com classificações distintas em diferentes pontos da cadeia, reduzindo a margem para interpretações divergentes que ocultem a natureza econômica real das transações.

6 Blockchain como Resposta ao Problema do Risco Sado

A discussão sobre registros distribuídos pode ser aprofundada a partir de uma pergunta direta: **como, especificamente, uma rede *blockchain* poderia ter mitigado o problema observado no caso Lojas Americanas?** A resposta envolve compreender tanto o potencial real da tecnologia quanto seus limites, evitando tanto o ceticismo precipitado quanto o entusiasmo ingênuo.

6.1 Conceitos Fundamentais

Blockchain é uma forma específica de tecnologia de registro distribuído (*Distributed Ledger Technology* — DLT) caracterizada por quatro propriedades centrais:

- **Imutabilidade:** uma vez registrado e validado, um bloco de transações não pode ser alterado retroativamente sem que toda a cadeia subsequente seja invalidada.
- **Compartilhamento:** múltiplos participantes mantêm cópias sincronizadas do mesmo registro, eliminando a figura do “livro único” sob controle exclusivo de uma das partes.
- **Carimbo temporal criptográfico:** cada evento é registrado com data, hora e assinatura digital verificáveis.

- **Consenso:** a inclusão de novos registros depende da concordância dos participantes, segundo regras pré-definidas.

Para aplicações corporativas, são particularmente relevantes as redes *permissioned* (permissionadas), como Hyperledger Fabric ou R3 Corda, nas quais apenas atores identificados e autorizados (empresa, bancos, fornecedores, auditores, reguladores) participam — distintas das *blockchains* públicas como Bitcoin ou Ethereum.

6.2 Aplicação Direta ao Caso

Considere-se a operação típica de risco sacado descrita na Seção 3. Hoje, ela gera registros independentes em três livros contábeis distintos: o da empresa compradora, o do banco financiador e o do fornecedor. Esses registros não conversam entre si, e nada impede — do ponto de vista operacional — que cada parte classifique a operação de forma diferente.

Em um modelo baseado em *blockchain* permissionada, a mesma operação seria registrada **uma única vez, em um livro compartilhado**, com visibilidade simultânea para todos os participantes autorizados. Os benefícios concretos seriam:

1. Eliminação da inconsistência entre registros

Tornar-se-ia operacionalmente inviável a empresa registrar a operação como “contas a pagar a fornecedores” enquanto o banco a registra como “crédito concedido”. Qualquer divergência apareceria automaticamente.

2. Rastreabilidade temporal completa

Cada etapa — emissão da nota fiscal, antecipação pelo banco, novação da dívida, ajustes de prazo — ficaria registrada em ordem cronológica imutável, com assinaturas digitais das partes envolvidas.

3. Auditoria contínua em substituição à *ex-post*

Auditores independentes e reguladores teriam acesso de leitura em tempo real, podendo configurar alertas automáticos para divergências, concentrações de risco ou padrões anômalos.

4. Aplicação de regras via *smart contracts*

As condições objetivas que caracterizam uma operação como dívida financeira (existência de banco intermediário, custo financeiro embutido, alongamento de prazo) poderiam ser codificadas em contratos inteligentes, gerando reclassificação automática e auditável quando os gatilhos fossem ativados.

5. Redução da discricionariedade gerencial

A possibilidade de “escolher” como classificar uma operação economicamente equivalente a dívida diminuiria substancialmente, pois a classificação seria função de critérios verificáveis e compartilhados.

6.3 Limites e Pré-Condições

A análise rigorosa exige reconhecer que *blockchain* não é solução autônoma. Sua eficácia depende de condições institucionais, regulatórias e de governança que precisam ser explicitadas.

Atenção: o que *blockchain* não resolve

1. **Não impede a inserção de dados incorretos na origem.** Se a operação for classificada erroneamente já no primeiro registro, a *blockchain* preservará essa classificação errada com perfeita integridade. O princípio “*garbage in, garbage out*” permanece válido.
2. **Não substitui o julgamento contábil.** Normas baseadas em princípios deliberadamente preservam zonas de avaliação profissional. Codificar essas regras em *smart contracts* exige decisões prévias — de natureza regulatória — sobre o que se torna automatizável.
3. **Não neutraliza fraude por conluio.** Se administração, contraparte bancária e auditor convergirem para um registro inadequado, a tecnologia preserva a fraude com a mesma fidelidade com que preservaria a verdade.
4. **Depende de obrigatoriedade regulatória.** Sem que reguladores (CVM, Banco Central, CPC) tornem o registro compartilhado mandatório para determinadas classes de operações, a adoção tende a ser fragmentada e insuficiente para gerar os efeitos sistêmicos desejados.
5. **Implica custos e desafios de implementação.** Padronização de dados, integração com sistemas legados (ERPs), governança da rede, definição de papéis e proteção de informações sensíveis são desafios técnicos e organizacionais não triviais.

6.4 Síntese: Tecnologia como Condição Facilitadora

A leitura mais equilibrada é a seguinte: a *blockchain* teria atuado, no caso Lojas Americanas, como um **poderoso redutor de assimetria de informação e de discricionariedade contábil**. A inconsistência entre o que a empresa registrava como “fornecedores” e o que os bancos registravam como “crédito concedido” provavelmente teria sido detectável anos antes da revelação pública — não pela genialidade da tecnologia, mas porque ela tornaria a divergência *visível* e *verificável* em tempo real.

Entretanto, a tecnologia é **condição facilitadora, não condição suficiente**. Sem mudança regulatória que torne o registro compartilhado obrigatório para operações de

risco sacado, sem padronização das regras de classificação contábil e sem incentivos alinhados entre os participantes da cadeia, a *blockchain* se reduz a uma infraestrutura tecnicamente sofisticada incapaz de produzir, isoladamente, os ganhos de transparência prometidos.

A lição central do caso, portanto, não é tecnológica, é **institucional**. A tecnologia ilumina o problema da governança, mas não a substitui.

7 Considerações Finais

Em síntese, o caso da Lojas Americanas não deve ser interpretado apenas como um evento isolado de inconsistência contábil, mas como um **exemplo paradigmático das fragilidades estruturais** presentes nos sistemas tradicionais de governança e *reporte* financeiro.

Sua análise oferece subsídios importantes para o desenvolvimento de modelos mais robustos de transparência corporativa, especialmente em ambientes caracterizados por elevada complexidade transacional e interdependência entre agentes econômicos. A discussão sobre *blockchain* apresentada na seção anterior reforça que soluções eficazes exigem articulação entre inovação tecnológica, evolução regulatória e maturidade institucional — nenhuma das três é dispensável.

Mais do que apontar culpados, o estudo do episódio convida gestores, conselheiros, auditores e reguladores a repensarem criticamente a arquitetura informacional sobre a qual repousa a confiança nos mercados de capitais.

A Código-Fonte do Contrato *Supply Chain Finance*

Este apêndice apresenta o código-fonte completo do contrato inteligente desenvolvido em **Solidity** (versão 0.8.20) para simular uma operação de risco sacado em rede blockchain. O contrato modela os quatro estados pelos quais uma fatura transita ao longo do ciclo de financiamento — *Criada*, *Aprovada*, *Financiada* e *Liquidada* — além do estado alternativo de *Rejeitada*.

A leitura desta seção, antes da execução prática descrita no Apêndice B, permite compreender a lógica de cada função e os controles de acesso impostos a cada participante da operação.

A.1 Visão Geral da Arquitetura

O contrato é organizado em sete blocos lógicos, descritos a seguir.

1. Estrutura de dados (`struct Invoice`)

Define o “registro digital” de uma fatura, contendo os campos essenciais à operação: identificador único, endereços do fornecedor, da empresa compradora e do banco financiador, valor da fatura, status atual e *timestamp* de criação. Cada fatura registrada na blockchain materializa um destes registros.

2. Enumeração de estados (`enum Status`)

Define formalmente os cinco estados possíveis de uma fatura: *Criada*, *Aprovada*, *Rejeitada*, *Financiada* e *Liquidada*. O uso de uma enumeração garante que apenas estados válidos possam ser atribuídos — impedindo, por exemplo, transições arbitrárias ou inconsistentes.

3. Variáveis de estado e mapeamentos

- **owner**: endereço do proprietário do contrato (responsável pela administração).
- **invoiceCounter**: contador sequencial de faturas emitidas.
- **invoices**: *mapping* que associa cada identificador a uma fatura.
- **authorizedBanks**: *mapping* que registra quais endereços estão autorizados a atuar como banco financiador — mecanismo análogo a uma autorização regulatória.

4. Eventos (`events`)

Os eventos são registros emitidos pela blockchain a cada transição relevante: criação, aprovação, rejeição, financiamento, liquidação e mudanças no rol de bancos autorizados.

Eles constituem o **rastro auditável** da operação — exatamente o tipo de log imutável que permite a auditoria contínua discutida no estudo.

5. *Modifiers* (controles de acesso reutilizáveis)

- `onlyOwner`: restringe a execução de uma função apenas ao proprietário do contrato.
- `invoiceExists`: verifica que o identificador informado corresponde a uma fatura existente, evitando consultas a registros inexistentes.

6. Funções de transição de estado

Estas funções implementam o ciclo de vida da fatura. Cada uma delas valida *quem* pode chamá-la e *em qual estado* a fatura precisa estar, garantindo que o fluxo só avance de forma legítima:

- `authorizeBank` / `revokeBank`: o proprietário cadastra ou remove bancos autorizados.
- `createInvoice`: o fornecedor cria a fatura. Validações: comprador válido, comprador diferente do fornecedor, valor positivo. Estado inicial: *Criada*.
- `approveInvoice`: somente a empresa compradora pode chamá-la, e somente enquanto a fatura está em *Criada*. Transiciona para *Aprovada*.
- `rejectInvoice`: alternativa ao `approveInvoice`. A empresa pode rejeitar a fatura, transicionando-a para *Rejeitada*.
- `financeInvoice`: somente endereços previamente autorizados podem chamá-la, e somente sobre faturas em *Aprovada*. Transiciona para *Financiada* e registra qual banco assumiu o crédito.
- `settleInvoice`: somente a empresa compradora pode chamá-la, e somente sobre faturas em *Financiada*. Transiciona para *Liquidada*, encerrando o ciclo.

7. Funções de consulta (*view*)

Funções de leitura, sem custo de gás e sem alterar o estado do contrato:

- `getInvoice`: retorna a estrutura completa da fatura.
- `getStatusText`: retorna o status atual em formato textual (“Criada”, “Aprovada” etc.), facilitando a leitura por humanos.

Conexão com o caso de estudo: cada *require* no contrato impede a execução de uma operação fora dos limites autorizados. Esta é a tradução técnica do conceito discutido na Seção 7: regras objetivas codificadas em *smart contracts* reduzem a discricionariedade gerencial e tornam a classificação contábil função de critérios verificáveis e compartilhados, e não de interpretação unilateral.

A.2 Código-Fonte Completo

O código a seguir corresponde ao contrato implantado no exercício prático descrito no Apêndice B.

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.20;
3
4  contract SupplyChainFinance {
5
6      // ----- ESTRUTURAS -----
7
8      enum Status { Criada, Aprovada, Rejeitada, Financiada, Liquidada
9          }
10
11     struct Invoice {
12         uint id;
13         address supplier;    // Fornecedor (Conta 1)
14         address buyer;      // Empresa (Conta 2)
15         address bank;        // Banco (Conta 3)
16         uint amount;
17         Status status;
18         uint createdAt;
19     }
20
21     // ----- ESTADO -----
22
23     address public owner;
24     uint public invoiceCounter;
25
26     mapping(uint => Invoice) public invoices;
27     mapping(address => bool) public authorizedBanks;
28
29     // ----- EVENTOS -----
30
31     event InvoiceCreated(uint indexed id, address indexed supplier,
32         address indexed buyer, uint amount);
33     event InvoiceApproved(uint indexed id);
34     event InvoiceRejected(uint indexed id);

```

```
33     event InvoiceFinanced(uint indexed id, address indexed bank);
34     event InvoiceSettled(uint indexed id);
35     event BankAuthorized(address indexed bank);
36     event BankRevoked(address indexed bank);
37
38     // ----- MODIFIERS -----
39
40     modifier onlyOwner() {
41         require(msg.sender == owner, "Only owner");
42         _;
43     }
44
45     modifier invoiceExists(uint _id) {
46         require(_id > 0 && _id <= invoiceCounter, "Invoice does not
47             exist");
48         _;
49     }
50
51     // ----- CONSTRUTOR -----
52
53     constructor() {
54         owner = msg.sender;
55     }
56
57     // ----- ADMINISTRACAO DE BANCOS -----
58
59     function authorizeBank(address _bank) public onlyOwner {
60         require(_bank != address(0), "Invalid bank");
61         authorizedBanks[_bank] = true;
62         emit BankAuthorized(_bank);
63     }
64
65     function revokeBank(address _bank) public onlyOwner {
66         authorizedBanks[_bank] = false;
67         emit BankRevoked(_bank);
68     }
69
70     // ----- PASSO 1: FORNECEDOR CRIA A FATURA -----
71
72     function createInvoice(address _buyer, uint _amount) public {
73         require(_buyer != address(0), "Invalid buyer");
74         require(_buyer != msg.sender, "Supplier cannot be buyer");
75         require(_amount > 0, "Amount must be positive");
76
77         invoiceCounter++;
78
79         invoices[invoiceCounter] = Invoice({
```



```
79         id: invoiceCounter,
80         supplier: msg.sender,
81         buyer: _buyer,
82         bank: address(0),
83         amount: _amount,
84         status: Status.Criada,
85         createdAt: block.timestamp
86     });
87
88     emit InvoiceCreated(invoiceCounter, msg.sender, _buyer,
89         _amount);
90
91     // ----- PASSO 2: EMPRESA APROVA OU REJEITA -----
92
93     function approveInvoice(uint _id) public invoiceExists(_id) {
94         Invoice storage inv = invoices[_id];
95         require(msg.sender == inv.buyer, "Only buyer can approve");
96         require(inv.status == Status.Criada, "Invoice not in Created
97             state");
98
99         inv.status = Status.Aprovada;
100         emit InvoiceApproved(_id);
101     }
102
103     function rejectInvoice(uint _id) public invoiceExists(_id) {
104         Invoice storage inv = invoices[_id];
105         require(msg.sender == inv.buyer, "Only buyer can reject");
106         require(inv.status == Status.Criada, "Invoice not in Created
107             state");
108
109         inv.status = Status.Rejeitada;
110         emit InvoiceRejected(_id);
111     }
112
113     // ----- PASSO 3: BANCO FINANCIA -----
114
115     function financeInvoice(uint _id) public invoiceExists(_id) {
116         require(authorizedBanks[msg.sender], "Not an authorized bank
117             ");
118
119         Invoice storage inv = invoices[_id];
120         require(inv.status == Status.Aprovada, "Invoice not approved
121             ");
122
123         inv.status = Status.Financiada;
124         inv.bank = msg.sender;
```

```
121         emit InvoiceFinanced(_id, msg.sender);
122     }
123
124
125     // ----- PASSO 4: EMPRESA LIQUIDA NO VENCIMENTO -----
126
127     function settleInvoice(uint _id) public invoiceExists(_id) {
128         Invoice storage inv = invoices[_id];
129         require(msg.sender == inv.buyer, "Only buyer can settle");
130         require(inv.status == Status.Financiada, "Invoice not
            financed");
131
132         inv.status = Status.Liquidada;
133         emit InvoiceSettled(_id);
134     }
135
136     // ----- CONSULTAS -----
137
138     function getInvoice(uint _id) public view invoiceExists(_id)
139         returns (Invoice memory)
140     {
141         return invoices[_id];
142     }
143
144     function getStatusText(uint _id) public view invoiceExists(_id)
145         returns (string memory)
146     {
147         Status s = invoices[_id].status;
148         if (s == Status.Criada) return "Criada";
149         if (s == Status.Aprovada) return "Aprovada";
150         if (s == Status.Rejeitada) return "Rejeitada";
151         if (s == Status.Financiada) return "Financiada";
152         if (s == Status.Liquidada) return "Liquidada";
153         return "Desconhecido";
154     }
155 }
```

B Tutorial de Execução do Contrato no Remix

Este apêndice apresenta um guia passo a passo para a execução prática do contrato apresentado no Apêndice A, utilizando a IDE Remix (<https://remix.ethereum.org>). A execução simula, de forma controlada, o fluxo de uma operação de **risco sacado** envolvendo três participantes: fornecedor, empresa compradora e banco financiador — estrutura central na discussão do caso Lojas Americanas.

A execução prática permite ao leitor visualizar concretamente como cada etapa da operação seria registrada em uma rede blockchain, evidenciando os ganhos de rastreabilidade e transparência discutidos ao longo do estudo.

B.1 Como funciona a interface do Remix

Em “Deployed Contracts” (área inferior do painel) o usuário encontrará:

- Um campo “**Select a function to interact with...**” (dropdown)
- Uma caixa “**Search functions...**” (busca)
- Uma lista de funções com bolinhas laranja

Para utilizar qualquer função: (i) clique na caixa **Search functions...**; (ii) digite o nome da função; (iii) clique nela na lista; (iv) preencha os campos de entrada (se houver) e clique no botão para executar.

△ **Antes de começar:** confirme que o campo **VALUE** (no painel de *deploy*, próximo a **ACCOUNT**) está em **0**. O envio de valor a funções não-*payable* resulta em reversão da transação.

B.2 Passo 1 — Anotar os endereços das contas

Por que fazer isso? Cada participante da operação (fornecedor, empresa e banco) precisa de um endereço único. Como o Remix gera contas de teste, é fundamental anotar a correspondência entre cada conta e seu papel, evitando confusões ao trocar de perfil durante a simulação.

No topo do painel **Deploy & Run Transactions**, abra o dropdown **ACCOUNT** e copie os endereços:

```
Conta 1 (DONO + FORNECEDOR): 0x5B3...
Conta 2 (EMPRESA):           0xAb8...
```

```
Conta 3 (BANCO): 0x4B2...
```

Utilize o ícone de cópia ao lado do dropdown para extrair cada endereço.

B.3 Passo 2 — Autorizar o banco

Por que fazer isso? O contrato exige que apenas bancos previamente autorizados possam financiar faturas, simulando o fato de que, no mundo real, somente instituições reguladas podem operar como banco. Esta etapa é executada uma única vez e somente o proprietário do contrato pode realizá-la.

Conta selecionada no topo: Conta 1

1. Na busca, digite: `authorize`
2. Clique em `authorizeBank`
3. No campo `address`, cole o endereço da **Conta 3**
4. Clique no botão para executar

Terminal verde = sucesso.

B.4 Passo 3 — Criar a fatura

Por que fazer isso? O fornecedor entregou mercadoria à empresa e emite uma fatura registrando essa dívida. No mundo real, equivale à emissão de uma nota fiscal com prazo de pagamento. Aqui, a fatura é registrada na blockchain de forma transparente e imutável.

Conta selecionada: Conta 1

1. Na busca, digite: `create`
2. Clique em `createInvoice`
3. Preencha:
 - `address (_buyer)`: endereço da **Conta 2**
 - `uint256 (_amount)`: 1000
4. Clique para executar

Verde no terminal = fatura criada.

B.5 Passo 4 — Empresa aprova a fatura

Por que fazer isso? A empresa compradora confirma que reconhece a dívida e se compromete a quitá-la no vencimento. Esta aprovação é o que oferece segurança ao banco para financiar a operação na etapa seguinte, pois assegura a existência de um pagador formalmente comprometido.

Troque para a Conta 2 no dropdown ACCOUNT

1. Na busca: `approve`
2. Clique em `approveInvoice`
3. No campo, digite: `1`
4. Execute

B.6 Passo 5 — Banco financia a fatura

Por que fazer isso? O banco antecipa o pagamento ao fornecedor (que recebe à vista, em vez de aguardar o prazo da fatura). Em contrapartida, o banco torna-se credor da empresa e cobrará dela no vencimento. **Esta é a essência econômica do risco sacado discutido no caso Lojas Americanas** — e o ponto exato em que a operação assume natureza de dívida financeira.

Troque para a Conta 3 no dropdown ACCOUNT

1. Na busca: `finance`
2. Clique em `financeInvoice`
3. Digite: `1`
4. Execute

B.7 Passo 6 — Empresa liquida a dívida

Por que fazer isso? Chegada a data de vencimento, a empresa paga o banco, encerrando o ciclo. Este é o passo final da operação: a dívida é quitada e a fatura passa ao status “Liquidada”.

Volte para a Conta 2 no dropdown ACCOUNT

1. Na busca: `settle`

2. Clique em `settleInvoice`
3. Digite: 1
4. Execute

B.8 Passo 7 — Verificar o resultado

Por que fazer isso? Funções de consulta (*view*) permitem verificar o estado atual do contrato sem alterá-lo e sem custo de gás. Este passo confirma que a fatura percorreu todo o ciclo até a liquidação — e ilustra a propriedade de **auditoria contínua** discutida na Seção 6 deste estudo.

Qualquer conta serve aqui (é apenas consulta).

1. Na busca: `getStatus`
2. Clique em `getStatusText`
3. Digite: 1
4. Execute

Será exibido logo abaixo: "Liquidada"

Ciclo completo! A fatura percorreu os estados: Criada → Aprovada → Financiada → Liquidada. Cada transição ficou registrada em ordem cronológica imutável, com assinaturas digitais das partes envolvidas — exatamente o tipo de rastreabilidade que torna inviável a classificação inconsistente da mesma operação em diferentes pontos da cadeia.

B.9 Resumo: qual conta usar em cada passo

Passo	Função	Conta
2	<code>authorizeBank</code>	1
3	<code>createInvoice</code>	1
4	<code>approveInvoice</code>	2
5	<code>financeInvoice</code>	3
6	<code>settleInvoice</code>	2
7	<code>getStatusText</code>	qualquer

△ **Em caso de erro:** observe a mensagem no terminal preto do Remix. As mensagens mais comuns indicam diretamente a causa (ex.: “*Only owner*”, “*Not an authorized bank*”, “*Invoice not approved*”), facilitando a depuração.

B.10 Conexão com o caso de estudo

A execução deste exercício demonstra, de modo concreto, três propriedades centrais discutidas ao longo do estudo:

1. **Registro único e compartilhado:** a mesma operação é registrada uma vez e visível a todos os participantes, eliminando a possibilidade de classificações divergentes entre os livros contábeis das partes.
2. **Rastreabilidade temporal:** cada transição de estado fica registrada em ordem cronológica imutável, permitindo reconstruir integralmente o histórico da operação.
3. **Auditoria contínua:** qualquer participante autorizado pode, a qualquer momento, consultar o estado atual e o histórico da fatura, substituindo o modelo *ex-post* pela verificação em tempo real.

Tais propriedades, conforme discutido na Seção 7, atuam como **condições facilitadoras** para a transparência corporativa — sem, contudo, dispensarem a necessidade de adequação regulatória, padronização contábil e governança institucional robusta.